컴퓨터공학부 졸업 작품 결과 보고서

졸업작품 팀명	사공사	지도교수	석승준 (인)
졸업작품 제목	어린이가 위험해요!		
주제분야 (융합 가능)	오픈 소스를 통한 인공지능 분석		
	졸업작품 주제 분야: 블록체인 응용, 오픈소스를 활용한 인공지능 응용 (영상인식 포함), 자율주행 응용시스템 개발, 빅데이터 분석시스템 및 시각화, 오픈소스를 활용한 IoT 시스템, 생체신호 분석 및 응용, 클라우드 및 컨테이너 시스템 응용, 게임 개발 (stand alone 방식 제외), SNS (응용) 개발, 웨어러블 컴퓨팅, 로봇 응용, 컴퓨터 보안		
포함 핵심 기술 (최소 2개 이상 포함)	☐ 오픈소스 활용 ☐ 알고리즘 설계 ☐ S/W공학 활용 ☑ UI 및 시각화 구현 ☑ 서버시스템(Linux & Unix) 구축 및 서버 프로그래밍 ☑ 데이터베이스 프로그래밍 ☑네트워크(통신)활용		

졸업 작품 요약

- 모델 개발하여 위험한 행동인가 아닌가를 판단하여 위험행동 분석을 함.
- 알림장 시스템 구현으로 프로그램의 주 사용자인 선생님이 위의 분석 결과물을 통해 각각 어린이에게 알림장을 쓸 수 있도록 함.

	학 번	이 름	연락처	팀원 역할
팀장	2022111757	신은진 (인)	01052439952	프로젝트 총괄 및 AI
팀원	2022111712	이예은 (인)	01030831136	DB 및 AI
팀원	2022111733	안채은 (인)	01021825795	AI 총괄
팀원	2022111813	장슬기 (인)	01083191601	Server
팀원	2022111296	황인정 (인)	01075600306	Raspberry Pi 및 Server

상기 내용과 같이 2022년도 컴퓨터공학부 졸업 작품 결과 보고서를 제출합니다.

2025년 6월 13일

졸업 작품 팀장: 신은진 신은진

1. 졸업 작품 개요

1) 개발 배경



그림 1. 하늘이 사건 요약

- 2025년 2월, 대전의 한 초등학교에서 발생한 사건은 많은 이들에게 충격을 줌. 당시 7세 김하늘 양이 교사 명재완 씨에게 흉기로 찔려 사망했으며, 이 사건은 '하늘이 사건'이라 부름.
- 가해 교사는 정신적 질환을 앓고 있었으며, 범행 당시 책을 준다며 학생을 시청각실로 데려간 후 목을 조르고 흉기로 찌른 것으로 밝혀짐.
- 이 사건은 어린이의 안전 문제에 대한 사회적 경각심을 높이는 계기가 되었으며, 안전 관리 필요성이 강조되었음.

2) 목적

- 자녀를 둔 학부모가 아이의 안전성을 실시간으로 확인하고, 기관 내 위험 요소를 사전에 감지할 수 있도록 돕는 것을 목표로 함.
- 교육 기관에서의 안전 관리 강화를 통해 아이들이 더욱 보호받을 수 있도록 하고 학부모가 자녀의 생활 환경을 알 수 있으며, 신뢰할 수 있는 정보를 제공받을 수 있도록 지원함.
- 이를 통해 학부모는 아이의 안전을 보다 적극적으로 관리할 수 있으며, 사회적으로도 어린이 보호에 대한 인식을 높이는 계기가 될 것이라 봄.

3) 필요성

- 어린이의 안전을 실시간으로 보호하기 위한 필수적인 시스템을 구축하는 것이 목적이기에 기능을 통해 행동 분석 기술을 활용하여 위험 행위를 감지하고 이를 바탕으로 교사는 신속하게 대응할 수 있음.
- 이는 단순한 사고 예방을 넘어 기관 내 안전 환경을 개선하고 교사들이 더욱 적극적으로 어린이를 보호할 수 있도록 지원하는 역할을 함.
- 학부모는 자녀가 안전한 환경에서 생활하고 있는지를 확인할 수 있어, 보다 신뢰할 수 있는 교육 환경을 조성하는데 기여함.
- 결과적으로 어린이 보호에 대한 사회적 인식을 높이고 안전한 미래를 만들어 나가는 데에 있어

중요한 역할을 할 것임.

2. 졸업 작품 목표

1) 최종 결과물의 설명 및 목표

- 카메라를 통해 실시간으로 인공지능 서버로 전송하여 인공지능 서버에서 분석 후 분석 정보를 서버에 전달.
- 정보(데이터)는 데이터베이스에 저장됨.

2) 평가기준

(1) 정량적 기준

- 정확도는 70% 이상이어야 함.

(2) 정성적 기준

- 회원가입, 로그인, 회원정보 수정 등 사용자 입력에 대한 응답 정보를 UI에 나타내야 함.
- 시스템 내 모듈 간 상호작용이 원활하게 진행되어야 하며 해당 응답을 손실 없이 처리해야함 .
- 개별 기능이 동작했을 때와 동일한 수준으로 시스템 전체 기능이 통합될 때도 적절하게 수행되어야 함. 사용자의 불편은 최소화해야 함.

2.1 프로젝트 요구사항 및 구현 기능

1) 사용자 측면

- 사용자에 따라 맞는 UI 환경이 제공됨.
- 회원가입 화면에서 계정 생성, 로그인 화면에서 로그인 할 수 있음.
- 선생님용 UI에선 담당 반에 속한 어린이의 알림장에 오늘 있었던 일과 다음 날의 준비물을 작성할 수 있으며, 어린이들의 전체적인 행동 분석을 볼 수 있어야 함.
- 학부모용 UI에선 선생님이 작성한 알림장을 확인할 수 있으며, 위험 행동 시 학부모가 확인할 수 있도록 영상과 사진을 제공받을 수 있어야 함. 또한, 하루 동안 발생한 각 행동 횟수를 알 수 있도록 통계를 볼 수 있어야 함.

2) 기술적 측면

- 라즈베리 파이 카메라는 흔들리지 않게 고정되어 있어야 함.
- 어린이의 행동을 위험 행동 분석을 통해 라벨링 작업을 거친 모델을 통해 분류되어야 함.
- 분류된 것으로 넘어짐, 뜀 등의 결과를 알 수 있어야 함.
- 위험한 행동인지 아닌지 확인 및 빈도를 확인할 수 있어야 함.
- 웹과 통신하기 위해 클라이언트는 Django 백엔드와 연동되며, 사용자의 입력은 HTTP 요청을 통해 서버로 전달되어야 함.
- 서버는 AWS EC2 인스턴스 상에서 동작 중인 Django 프레임워크를 통해 요청을 처리하고, 필요한 데이터는 MongoDB에서 조회하여 응답으로 반환되어야 함.

2.2 컴퓨터공학 핵심 기술에 대한 졸업 작품 반영 내용 기술 (2개 이상)

1) YOLO v8을 활용한 객체 인식

(1) 실행 환경

- Anaconda에서 진행하여 jupyter notebook에 가상환경을 다운받아서 진행했다.
- python 버전은 3.8, CUDA 는 12.n 버전의 최신버전을 사용했다.

(2) AI 훈련

- 모델을 제작하기 위해 클래스를 정하여 그에 맞는 이미지와 영상을 구해 라벨링을 한다.
- Roboflow를 이용해 라벨링을 진행하여 train, test, valid 데이터를 나누어 학습시키고 모델을 제작한다. 다음으로 모델 코드를 실행코드로 옮겨서 진행한다.

(3) AI 분류

- 모델을 사용하여 객체 인식을 한 이후 동일 객체 인식을 위해 strongSORT를 이용해서 인식한다.
- 이후 분류된 결과를 .mp4와 .csv파일로 저장하여 서버로 전송한다.

2) UI 구현

(1) HTML

• 웹페이지의 구조와 콘텐츠를 정의하기 위해 HTML5를 사용함.

• 시맨틱 태그(header, nav, main, section 등)를 활용하여 구조적인 웹페이지를 구성함.

• 사용자 입력을 위한 폼(form) 요소(input, select, textarea, button 등)를 통해 회원가입, 로그인, 자녀 정보 입력 등 주요 기능을 구현함.

• Django 템플릿 문법({{ 변수 }}, {% if %}, {% for %} 등)을 HTML 내부에 적용하여 서버에서 전달받은 데이터를 웹페이지에 동적으로 출력함.

3) Raspberry Pi 사용

(1) 임베디드 IoT 시스템 구현

- Raspberry Pi 4 + HQ Camera 모듈 장착·제어(Picamera2 API)
- SSH 원격 운영·systemd 서비스로 부팅 즉시 자율 동작 설정
- 저전력 엣지 노드 특성에 맞춰 해상도·FPS 동적 조절

(2)실시간 네트워크 스트리밍 프로그래밍

- OpenCV로 프레임 JPEG → Base64 인코딩, WebSocket(JSON) 실시간 송신
- FastAPI WebSocket 서버가 즉시 복원·표시 → 지연 최소화(≈ 300 ms)
- 네트워크 장애 시 자동 재연결·재시도 로직 삽입

(3) 멀티미디어 인코딩 & 파일 전송 자동화

- libcamera-vid(RPi)·FFmpeg로 5 분 단위 H.264 → MP4 무손실 변환
- multipart/form-data POST 방식으로 AI 서버에 비동기 업로드 + 스레드풀
- 업로드 성공 시 자동 삭제 / 실패 시 보존 폴더 분리 → 데이터 무결성 확보

4) 서버 시스템 구축 및 서버 프로그래밍

- AWS EC2 인스턴스를 기반으로 서버 환경을 구축하였으며, 운영체제로는 Ubuntu 24.04 LTS를 사용함.
- 백엔드 프레임워크로 Django를 활용하였고, 웹 서버는 Nginx를 통해 배포 환경을 구성함.
- Django의 WSGI 인터페이스를 기반으로 애플리케이션을 실행하였으며, 웹 서버는 Nginx를 사용하여 정적 자산 처리 및 기본 라우팅을 수행함.

5) Mongo DB를 활용한 데이터베이스 프로그래밍

- MongoDB는 오픈소스 NoSQL 데이터베이스 시스템으로, 다양한 형태의 데이터를 효율적으로 저장하고 관리할 수 있음.
- 로젝트에서는 데이터의 목적에 따라 컬렉션을 분리하여 사용하며, 각 컬렉션은 필요한 필드를 정의하고, AI 모델 또는 사용자 인터페이스(UI)로부터 전달된 데이터를 저장함.

6) 인공지능 서버 시스템 구축 및 서버 프로그래밍

- AWS EC2 인스턴스를 기반으로 한 Amazon Linux 서버환경을 사용함.
- 백엔드 프레임워크로 Flask를 이용함.

2.3 현실적 제한요소

현실적 제한요소	반영 비율	반영 내용	비고
성능	70%	• 서버는 오류 없이 상시 구동되어야 한다. • 회원(유저) 정보는 안전하게 저장 및 관리되어야 하며, 데이터베이스 조회 결과가 사용자에게 일관되고 정확하게 전달되어야 한다. • 라즈베리파이 카메라에서 촬영된 영상은 실시간으로 AI 서버에 안정적으로 전송되어야 한다.	
편의성	10%	• 사용자에게 직관적이고 사용하기 편한 UI와 각 사용자의 역할에 따라 다른 UI를 제공해야 한다. • 사용자가 작성한 정보와 내용은 바로 보일 수 있도록 해야 한다. • 행동 분석의 결과가 한눈에 들어올 수 있도록 해야한다. • 시스템 내 각족 알림 및 안내 메시지는 명확하고 이해하기 쉽게 제공되어야 한다. • 주요 기능은 복잡하지 않고 사용자가 최소한의 단계로 원하는 작업을 완료할 수 있도록 설계해야 한다,	
신뢰성	10%	• 정보 처리에 대한 결과값을 서버에 전달해주며 데이터 송수신 간 데이터 손실이 없어야 한다. • 기관 내 일어난 일이 꾸밈없이 사실적이어야 한다.	

		• 사용자가 입력한 정보는 데이터베이스에 저장되어야 한다. • 오로지 관리자만 정보를 확인할 수 있으며 다른 이는 확인할 수 없어야 한다. • 사용자가 입력한 정보(비밀번호)는 해시화되어 데이터베이스에 저장되어야 한다. • 사용자가 탈퇴하고자 할 경우 한 번 더 확인을 거쳐야 한다. • 로그아웃을 한 뒤 로그인 화면에서 뒤로가기 기능이 되지 않도록 한다.	
실용성	10%	• 위험한 행동인지 아닌지 분석함으로써 어린이의 위험도를 확인할 수 있어야 한다.	

2.4 졸업 작품 최종 예상 목표

1) 프로젝트 목표

- 카메라는 설치된 공간에서 무중단 상태로 제대로 작동해야 함.
- AI 모델은 카메라를 통해 들어오는 실시간 영상으로 아이들의 위험 행위 분석을 수행함.
- 서버는 행위 분석 결과와 유저/아이들 정보를 웹페이지로 신속하고 손실 없이 보내 사용자가 정확한 정보를 확인할 수 있도록 함.

2) 최종 결과물

- 사용자는 유저ID, 비밀번호, 이름, 전화번호, 담당 반(선생님의 경우)을 통해 계정을 생성하며, 비밀번호는 해시화하여 안전하게 데이터베이스에 저장됨.
- 데이터베이스에 저장되는 유저ID는 중복되지 않도록 함.
- 로그아웃 시엔 캐시가 무효화되어 로그아웃 전의 페이지로 돌아갈 수 없도록 함.
- 회원 탈퇴 시 실수로 인한 탈퇴를 방지하기 위해, 추가 확인 절차를 통해 최종 의사를 다시 한 번 확인하도록 함.
- 로그인 후 각 사용자에게 필요한 데이터를 빠르고 정확하게 불러와 원활한 사용자 경험을 제공함.
- 알림장 내용이나 자녀 정보는 저장하는 즉시 반영되어야 하며, 사용자가 체감할 수 있는 지연 없이 실시간으로 적용됨.
- 선생님과 학부모는 위험 행동을 빠르게 확인할 수 있음. 선생님은 위험 행동에 대한 피드백이나 하루 있었던 일을 토대로 쓴 알림장을 학부모에게 전달할 수 있음.
- 데이터베이스에 저장되거나 조회될 때, 데이터의 손실 없이 원본 그대로의 정확한 상태로 유지되어야 한다.
- 라즈베리파이 카메라로 영상을 촬영할 때, 영상은 끊김 없이 안정적으로 촬영되어야 하며, 분석에 지장이 없도록 일정 수준 이상의 화질을 유지함.
- 실시간 카운팅 및 결과물 반영을 위해 10초마다 저장된 데이터를 파일에 업데이트함.
- 객체 인식 시, 객체가 움직임이 감지되면 해당 움직임을 반영하여 바운딩 박스를 동적으로 조

정함.

- 데이터 파일 저장 시 프로그램이 종료되는 순간의 데이터까지 마지막으로 저장할 수 있도록 함.

3) 습득 또는 학습 가능한 기술

- AWS EC2 인스턴스를 사용하며 아마존 클라우드에 대한 개념을 학습할 수 있음.
- Django 프레임워크를 사용하며 MTV 아키텍처와 웹 서버 구축, URL 라우팅, 템플릿 렌더링 등 웹 백엔드 개발의 전반적인 흐름을 이해할 수 있음.
- Django를 활용한 사용자 인증 및 세션 관리 및 HTTP 통신 구현을 통해 서버와 클라이언트 간의 요청/응답 구조에 대한 실질적인 이해를 할 수 있으며, 클라이언트와 서버 간 JSON 데이터를 송수신하는 방식에 대한 이해와 활용 능력을 습득할 수 있음.
- 프론트엔드 영역에서는 HTML, CSS, JavaScript를 활용한 웹 화면 구성 경험을 통해 전체적인 웹 개발 흐름을 학습할 수 있음.
- MongoDB에서 컬렉션과 데이터의 각 필드와 타입을 설계할 수 있음. 또한, MongoDB의 고유한 ObjectId를 통해 필요한 정보를 저장하거나 가져올 수 있음.

3. 수행 내용 및 주요성과

3.1 시스템 구조 개요

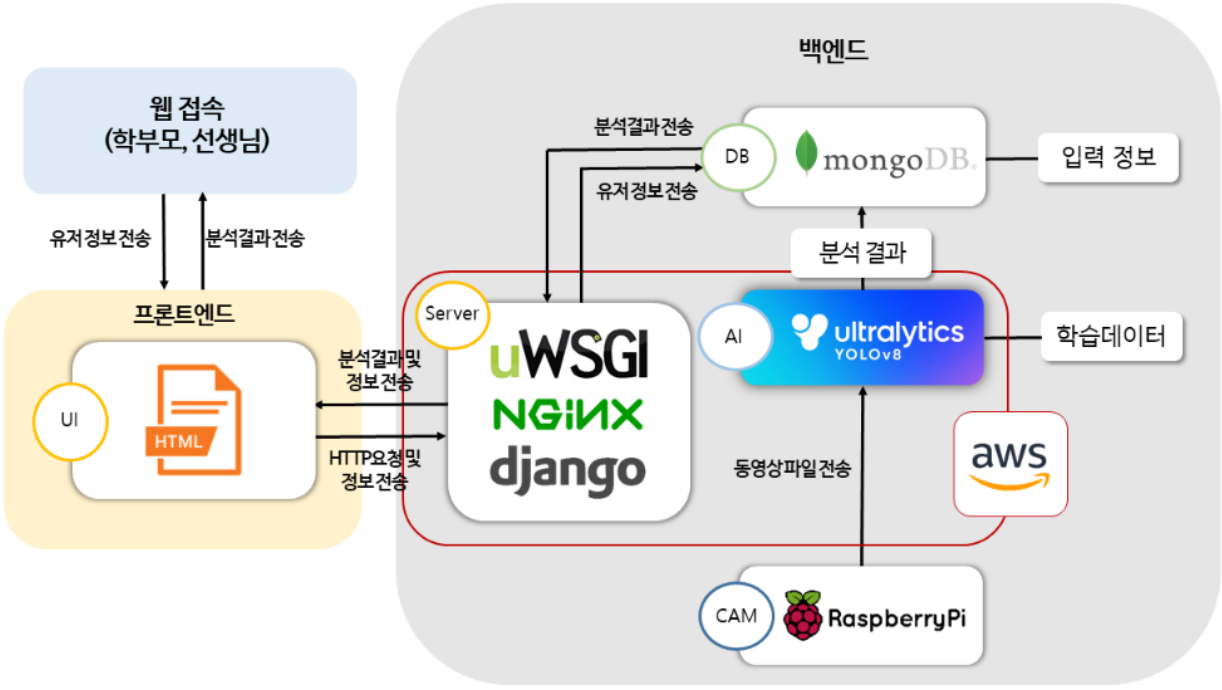


그림 2. 전체 시스템 구조도

- 전체 시스템 구성도는 그림 2와 같음
- HTML과 서버 간에 HTTP 요청을 통해 회원가입, 로그인을 수행함.
- Django 내부에 HTML 템플릿이 존재하지만, 사용자 브라우저에서 렌더링 되므로 구조도에선 프론트엔드 영역에 포함시킴.
- YOLOv8과 웹서버는 AWS EC2 인스턴스 안에 존재함.

3.2 시스템 구조도 및 알고리즘

1) AI_server 통신

구축한 리눅스 기반 aws 서버로 ai, 라즈베리 파이 측과 데이터 송수신을 진행함.

(1) 리눅스 기반 서버 구축 및 관리

- Python 기반 AI 서버를 Ubuntu 리눅스 환경에 Flask로 구축함.
- 서버는 외부와 통신할 수 있도록 방화벽 설정 및 포트(예: 5000) 허용을 진행함.
- 사용하지 않는 영상 파일 삭제 및 로그 저장 경로 설정 등 유지관리 작업 수행함.
- MongoDB Atlas에 접근할 수 있도록 .env 또는 시크릿 키를 활용해 보안 처리함.

(2) 촬영 영상 수신 및 행동 분석 결과 전송

- Flask 기반의 AI 분석 서버는 라즈베리파이 전송되는 .mp4 형식의 영상 데이터를 /upload 라

우트를 통해 수신함.

- 수신된 영상은 서버 내 지정된 경로(예: uploads/)에 저장되며, 저장 직후 predict.py 모듈이 해당 영상을 분석 요청받음.
- predict.py는 영상에서 일정 간격으로 프레임을 추출하고, 사전 학습된 AI 모델을 통해 각 프레임에 대한 행동을 분류함.
- 분석 결과는 CSV 파일로 저장되며, 이후 save_result.py 모듈을 통해 MongoDB Atlas에 전송되어 ai_results 컬렉션에 저장됨.
- Flask 서버는 분석 완료 여부 및 저장 상태를 로그로 기록하며, 향후 학부모/교사에게 전달될 정보를 웹 서버 또는 프론트엔드로 전달할 수 있도록 연계함.

2) 웹서버 통신

- Ubuntu 기반의 AWS EC2 인스턴스에서 Django 서버를 실행하여, 공인 IP를 통해 사용자가 웹 애플리케이션에 접속하고, 서버 내부에서는 데이터베이스와 연결하여 요청을 처리하도록 구성함.

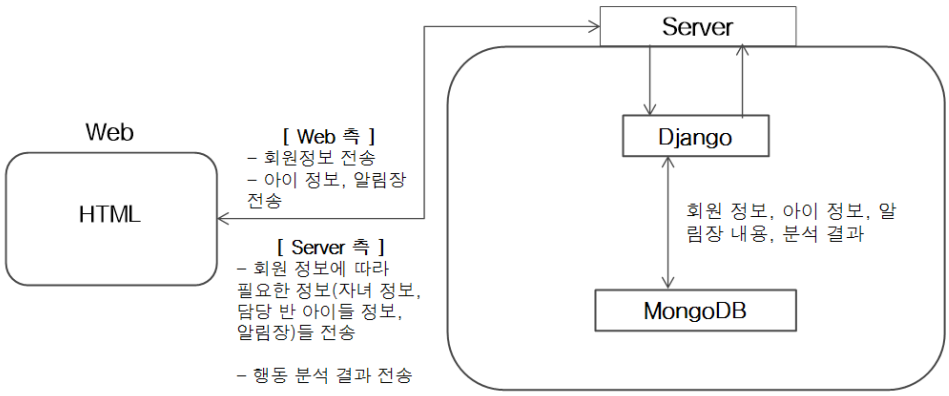


그림 3. 웹서버 통신

(1) Ubuntu 기반 AWS 서버 구축 및 관리

- IP 주소는 AWS EC2의 탄력적 IP를 통해 웹을 어디서든 접속할 수 있도록 함. 해당 IP를 이용해 SSL 인증서도 발급받고, DNS도 설정함.
- AWS EC2 보안 그룹을 통해 접속이 가능한 포트를 제한 하여 보안을 강화함.
- MongoDB로 데이터베이스를 생성함. 데이터베이스를 통해 회원(유저) 정보를 모두 관리하며, 추가적으로 역할에 따른 학부모, 선생님 컬렉션과 아이들 컬렉션, 알림장과 분석 결과를 저장할 수 있는 컬렉션이 존재함.
- 컬렉션의 데이터는 모두 각각의 고유한 ObjectId가 존재하여 해당 id를 이용해 연관된 데이터를 찾을 수 있도록 함.

(2) 회원(유저) 정보, 아이 정보, 알림장 저장 및 검색

• HTTP POST 방식을 이용하여 views.py와 웹페이지를 통해 회원 정보를 MongoDB에 저장함. 저장 요청을 받으면 고유의 ObjectId나 관련 데이터로 컬렉션을 찾아서 따로 저장하고 성공 여부를 전달함.

• 학부모로 로그인 할 경우 해당 학부모의 고유 Id가 포함된 어린이 데이터들을 찾아 검색해 웹에 띄워주고, 해당 어린이의 고유 Id가 포함된 현재 날짜의 알림장 내용과 행동 분석 결과도 함께 전달해 줌.

3) UI

사용자의 역할(선생님, 학부모)에 따라 다른 UI를 제공하도록 설계하고 구현함. HTTP 통신을 이용하여 회원(유저) 정보와 분석 결과를 요청하여 Django를 통해 응답을 받음.

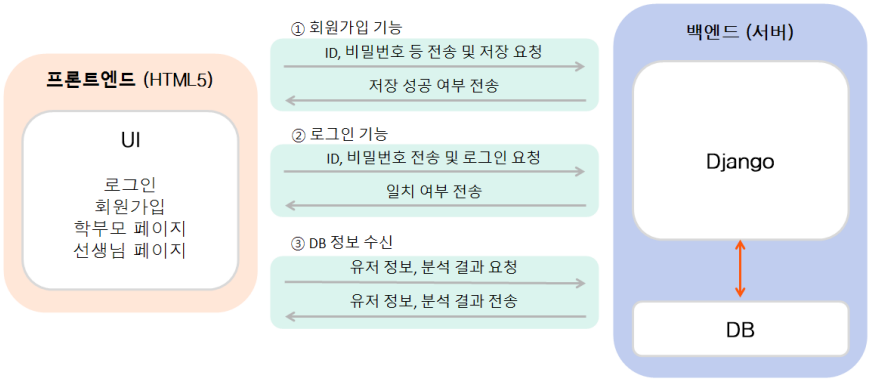


그림 4. HTML5와 서버 통신

(1) 회원정보 관리

① 회원가입

• '회원가입' 화면에서 사용자 ID(username), 비밀번호, 이름, 전화번호, 역할(선생님, 학부모) 등을 받음. 선생님일 경우엔 담당하는 반 이름도 함께 받게 함. 이후에 HTTP 통신을 통해 Django를 거쳐 데이터베이스에 저장함. 비밀번호를 저장할 땐, Django User모델의 기본 'PBKDF2 해시 알고리즘'으로 비밀번호를 변환함.

② 로그인

• '로그인' 화면에서 역할, 사용자 ID(userame), 비밀번호를 데이터베이스에 저장한 값과 비교함과 동시에 Django의 인증(Authenticaton) 시스템을 사용하여 더욱 확실하게 성공 여부를 결정함. 또한, Django에서 제공하는 login() 함수를 사용하여 로그인 상태임을 나타냄.

(2) 데이터베이스 정보 전달 및 수신

• 사용자가 로그인함과 동시에 유저의 역할에 맞는 연관된 데이터베이스 정보를 함께 가져와 홈페이지에 띄울 수 있도록 함. 홈페이지 내에서 저장할 땐 해당 유저의 고유 ID를 함께 저장하여 서로 연관되도록 해 정보를 불러올 때 혼동되지 않도록 함.

4) Raspberry Pi

(1) CameraCapture - 프레임 획득

- Picamera2 로 해상도 640×480·FPS 30 설정 후 capture_array() 반복 호출.
- 획득 프레임을 FrameEncoder 큐로 전달.

(2) FrameEncoder - JPEG/베이스64 인코딩

- OpenCV imencode() 로 프레임 → JPEG 압축.
- base64.b64encode() 로 문자열 변환.
- JSON payload(type: IMAGE) 구성 후 WebSocketSender 호출.

(3) WebSocketSender - 실시간 전송
- websocket-client 로 ws://EC2_IP:5001/ws/video 연결
- 연결 끊김 감지 → 1 초 간격 재시도
- 지연 목표: ≤ 500 ms

(4) SegmentRecorder - 5 분 H.264 → MP4
- libcamera-vid(혹은 ffmpeg) 로 300 초 녹화(H.264)
- 녹화 종료 후 ffmpeg copy 코덱으로 무손실 MP4 변환
- 완료 파일을 UploadQueue에 push

(5) UploadWorker - AI 서버 업로드
- 스레드 2개, 큐에서 MP4 pop → multipart/form-data POST (video 필드)
- 3회 재시도(백오프 2→4→8 초). 성공 시 파일 삭제, 실패 시 보존 폴더 이동

3.3 시스템 하드웨어 설계

1) 서버
- 클라우드 플랫폼: AWS EC2
- 인스턴스 타입: t2.micro
- 운영체제: Ubuntu 24.04.2 LTS
- 커널 버전: GNU/Linux 6.8.0-1026-aws x86_64
- 주요 설치 패키지: Django, Nginx, Wsgi, Python, pymongo 등

2) 인공지능 서버
- 클라우드 플랫폼 : AWS EC2
- 인스턴스 타입 : t2.micro
- 운영체제 : Amazon Linux 2023
- 커널 버전 : Linux 6.1.134-152.225.amazon2023.x86_64
- 주요 설치 패키지 : Flask, pymongo, PyTorch 등

3.4 소프트웨어 모듈 상세 설계

1) Raspberry Pi

모듈명	CameraCapture

입력 자료	• 카메라 설정값 - 해상도 640×480 - FPS 30 - 포맷 RGB888 • 종료 플래그 stop_recording 출력 자료 레임(frame: NumPy BGR 배열) • 상태/오류 로그
출력 자료	• 실시간 프레임(frame: NumPy BGR 배열) • 상태/오류 로그
기능	1. Picamera2 초기화 및 미리보기 구성 2. 루프에서 capture_array() 로 프레임 획득 3. 획득 프레임을 FrameEncoder 큐로 전달 4. 종료 요청 시 카메라 안전 종료 호출모듈 rameEncoder → 이후 WebSocketSender 로 전달
호출모듈	(피호출) FrameEncoder → 이후 WebSocketSender 로 전달
모듈의 상세 설계	pseudo function init_camera(w,h,fps): cam ← Picamera2() cam.preview_configuration.main.size = (w,h) cam.preview_configuration.main.format = “RGB888“ cam.configure(“preview“) cam.start() return cam function capture_loop(cam): while not stop_flag: frame = cam.capture_array() queue.put(frame) # → FrameEncoder function close_camera(cam): cam.stop()<br

2) Server, DB

모듈명	settings.py 중 ALLOWED_HOSTS
입력 자료	없음.
출력 자료	없음.
기능	Django가 실행되기 위한 설정 파일 중에서 외부에서 접속을 위한 아이피 주소. 작성된 주소로만 Django/Nginx로 접속할 수 있음.
호출모듈	
모듈의 상세 설계	ALLOWED_HOSTS = ['13.209.113.91', 'api.ip.pn', 'www.dangerchildren.site', 'dangerchildren.site', 'ip-172-31-38-155.ap-nortexhortexh-2.compute.internal']

모듈명	views.py/add_users
입력 자료	회원 정보(id, pw, 역할, 이름, 전화번호, 반(선생님일 때))
출력 자료	DB에 정보 저장
기능	웹에서 POST로 전달하는 회원 정보를 받아 DB에 저장하도록 하는 코드를 동작시킴.
호출모듈	MongoDB에 접속하는 views.py와 models.py

모듈의 상세 설계	# POST 데이터 받기
	username = request.POST.get('username')
	password = request.POST.get('password')
	role = request.POST.get('role')
	name = request.POST.get('name')
	contact = request.POST.get('phoneNumber')
	if role == "teacher":
	  classroom = request.POST.get('classroom')
	(이하 생략)
	
	# 데이터 생성
	data = {
	  "username": username,
	  "password": user.password, # 해시된 비번 저장
	  "role": role,
	  "name": name,
	  "createdAt": datetime.now()
	}
	
	# mongoDB users 컬렉션에 저장 저장
	users_collection.insert_one(data)
	
	# 각 유저 역할에 따른 정보도 추가로 저장
	if role == "parent":
	  parents_collection.insert_one({
	  "name": name,
	  "contact": contact,
	  "children_ids": []
	  })
	elif role == "teacher":
	  teachers_collection.insert_one({

	“name“: name, “contact“: contact, “classroom“: classroom })

모듈명	views.py/login_user
입력 자료	회원 정보(id, pw, 역할)
출력 자료	DB에 존재하는 회원 정보
기능	웹에서 POST로 전달하는 회원 정보를 받아 DB에 존재하는지 확인 후 역할에 따른 페이지로 리디렉션함. 리디렉션 하면서 필요한 정보들은 세션에 저장해둠.
호출모듈	MongoDB에 접속하는 views.py와 models.py

모듈의 상세 설계	# POST 데이터 받기
	username = request.POST.get('username')
	input_password = request.POST.get('password')
	role = request.POST.get('role')
	(이하 생략)
	
	# Django 인증 시스템으로 먼저 인증 시도
	user = authenticate(request, username=username, password=input_password)
	
	# 후 MongoDB에서 역할 등 추가 정보 확인
	# 데이터 조회 (username, password가 일치하는 데이터가 있는지 확인)
	user_data = users_collection.find_one({“username“: username, “role“: role})
	
	login(request, user) # Django 인증 시스템에 로그인 처리 (세션 생성)
	
	# role에 따라 리디렉션
	if role == “parent“: # 부모님
	# 로그인한 부모 사용자와 일치하는 부모 정보만 찾기
	parent_data = parents_collection.find_one({“name“: user_data['name']})
	# 세션에 저장 (조건: 존재하고 children_ids가 비어있지 않으면)
	if parent_data and 'children_ids' in parent_data:
	request.session['children_ids'] = [str(cid) for cid in parent_data['children_ids']]
	else:
	request.session['children_ids'] = [] # 기본값
	return redirect('showParent') # 학부모 페이지로 이동
	elif role == “teacher“: # 선생님

	teacher_data = teachers_collection.find_one({“name“: name}) if teacher_data and 'classroom' in teacher_data: request.session['classroom'] = teacher_data.get('classroom') return redirect('teachers_page') # 선생님 페이지로 이동

모듈명	views.py/showParents
입력 자료	없음
출력 자료	자녀 정보, 학부모 이름 등
기능	로그인 페이지에서 학부모라면 리디렉션 되는 페이지. 학부모의 따라 다른 자녀를 보여주기 위한 로직 구현됨.
호출모듈	MongoDB에 접속하는 views.py와 models.py

모듈의
상세 설계
	# 로그인 세션에서 부모 이름 가져오기
	parent_name = request.session.get('name')
	parent_doc = parents_collection.find_one({“name“: parent_name})
	
	# 로그인된 학부모의 자녀 정보 가져오기
	for cid in parent_doc.get(“children_ids“, []):
	child = children_collection.find_one({“_id“: ObjectId(cid)})
	if child:
	birthdate = child.get(“birthdate“)
	# 생일로 만 나이 계산
	if birthdate and isinstance(birthdate, datetime):
	age = calculate_age(birthdate.date()) # datetime.date로 변환
	else:
	age = “정보 없음“
	
	parent[“children“].append({
	“id“: str(child[“_id“]),
	“name“: child[“name“],
	“age“: f“{age}세“ if isinstance(age, int) else age,
	})

모듈명	views.py/teachersPage
입력 자료	없음
출력 자료	담당 반 어린이들 정보, 선생님 이름 등
기능	로그인 페이지에서 선생님이라면 리디렉션 되는 페이지. 선생님이 담당한 반에 따라 해당하는 어린이들을 보여주기 위한 로직 구현.

호출모듈	MongoDB에 접속하는 views.py와 models.py

모듈의
상세 설계
	# 로그인 세션에서 정보 가져오기
	teacher_name = request.session.get('name')
	teacher_doc = teachers_collection.find_one({"name": teacher_name})
	
	# 선생님이 담당하는 어린이 목록 가져오기
	for child in children_collection.find({"classroom":
	teacher_doc.get("classroom")}):
	# 아이들 나이 계산 추가
	birthdate = child.get("birthdate")
	# 생일로 만 나이 계산
	if birthdate and isinstance(birthdate, datetime):
	age = calculate_age(birthdate.date()) # datetime.date로 변환
	else:
	age = "정보 없음"
	
	teacher["children"].append({
	"id": str(child["_id"]),
	"name": child["name"],
	"age": f"{age}세" if isinstance(age, int) else age,
	"parent_id": str(child.get("parent_id"))
	})

모듈명	views.py/logout
입력 자료	없음
출력 자료	없음
기능	Django가 해당 유저의 로그아웃 사실을 알고, 세션 만료를 시켜 로그아웃 후 뒤로가기가 되지 않도록 하기 위함.
호출모듈	MongoDB에 접속하는 views.py와 models.py
모듈의 상세 설계	request.session.flush() # 세션 데이터 초기화 request.session.clear_expired() # 만료된 세션 데이터 제거 logout(request) # 장고의 User 인증 기능 중 하나 return redirect('login_user')

모듈명	views.py/add_child
입력 자료	아이들 정보(이름, 생일, 반)
출력 자료	DB에 정보 저장
기능	학부모가 자녀들을 추가할 때 웹에서 POST로 전달하는 아이의 데이터를 데이터베이스에 저장됨.
호출모듈	MongoDB에 접속하는 views.py와 models.py

모듈의 상세 설계	# 웹페이지에서 입력된 값 들고오기
	parent_id = request.POST.get('parent_id')
	child_name = request.POST.get('childname')
	birthdate = request.POST.get('birthdate')
	classroom = request.POST.get('classroom')
	
	# 데이터 생성
	data = {
	"name": child_name,
	"birthdate": birthdate,
	"parent_id": ObjectId(parent_id), # 홈페이지에서 session으로 가져온 값을 ObjectId 형태로 저장
	"classroom": classroom
	}
	
	# mongoDB에 어린이 데이터 저장
	inserted_child = children_collection.insert_one(data)
	
	# 그리고 생성된 어린이 고유 id를 부모님 컬렉션에 업데이트
	parents_collection.update_one(
	{"_id": ObjectId(parent_id)},
	{"$push": {"children_ids": child_id}} # child_id
	)
	return redirect('showParent') # 다시 부모 페이지로

모듈명	views.py/writeNotice
입력 자료	알림장 내용
출력 자료	DB에 정보 저장

기능	웹에서 POST로 전달하는 각 아이들의 알림장의 내용을 저장함.
호출모듈	MongoDB에 접속하는 views.py와 models.py

모듈의 상세 설계	content = request.POST.get('content')
	child_id = request.POST.get('child_id') # 열려있는 알림장 주인(어린이 ID)
	
	date = datetime.now(timezone.utc)
	format_date = datetime(date.year, date.month, date.day)
	notice = {
	“child_id“: child_id,
	“content“: content,
	“date“: format_date, # 현재 시간(UTC 기준으로 저장)
	“teacher_id“: teacher_id,
	“classroom“: classroom
	}
	# 알림장 DB에 저장
	notice_collection.insert_one(notice)

모듈명	views.py/showNotice_cont
입력 자료	없음
출력 자료	DB에 저장된 자신의 자녀의 각각의 알림장 내용
기능	DB에 저장되어 있는 자녀의 알림장 내용을 가져옴. 각 자녀의 고유의 ID를 넘겨받아 해당 아이에 맞는 알림장 내용 출력
호출모듈	MongoDB에 접속하는 views.py와 models.py

모듈의 상세 설계	kst = pytz.timezone('Asia/Seoul')
	# 오늘 날짜 검색하기
	today = datetime.now(kst)
	# 자정으로 초기화 + 타임존 유지
	start = today.replace(hour=0, minute=0, second=0, microsecond=0)
	end = start + timedelta(days=1)
	
	# 알림장 컬렉션에서 알맞는 결과를 얻기 위한 쿼리
	querys = {
	'child_id': id,
	‘date': start.replace(tzinfo=None)
	}
	cont = notice_collection.find_one(querys, {'content': 1, '_id': 0}) # 쿼리에

	맞는 결과가 cont에 저장됨
	
	# 행동 분석 결과 컬렉션에서 알맞은 결과를 얻기 위한 쿼리
	query = {
	'child_id': id,
	'timestamp': {
	'$gte': start,
	'$lt': end
	}
	}
	# 쿼리에 맞는 것들을 모두 들고옴
	results_collection.find(query)
	total_res = results_collection.count_documents(query) # 오늘 하루의 행동 분석 갯수
	event_counts = {} # 각 행동들의 개수 저장하기 위한 변수 지정
	
	ALL_EVENTS = [“standing“, “sitting“, “walking“, “running“, “playing“, “fighting“, “falldown“]
	# 기본적으로 모든 행동에 대한 값을 넣어둠. 기본 값은 0으로 설정
	event_counts = {event: 0 for event in ALL_EVENTS}
	
	# 쿼리에 맞는 값을 들고와서 해당하는 행동들의 개수를 +1 해주기
	for doc in results_collection.find(query):
	event_type = doc.get('event_type')
	if event_type in event_counts:
	event_counts[event_type] += 1
	
	# 다시 홈페이지로 Json으로 보내줄 때 사용할 변수
	sum_data = {}
	if cont and 'content' in cont:
	sum_data['content'] = cont['content']
	
	if total_res > 0:
	sum_data['total_res'] = total_res
	else: sum_data['total_res'] = 0
	
	sum_data['event_counts'] = event_counts

모듈명	views.py/withdrawalUser
입력 자료	없음
출력 자료	DB에 저장된 자신의 자녀의 각각의 알림장 내용

기능	웹에서 POST 되어 현재 로그인 되어있는 유저의 아이디를 이용해서 탈퇴함. 학부모와 탈퇴할 땐 자녀들도 같이 삭제하며, 각 어린이들의 행동 분석 결과 또한 삭제함. 유저 컬렉션에도 삭제됨.
호출모듈	MongoDB에 접속하는 views.py와 models.py

모듈의 상세 설계	# 유저 컬렉션에서 삭제
	delete_user = users_collection.delete_one({'username': username})
	# 해당 이름이 학부모 컬렉션에 존재한다면
	if parents_collection.find_one({'name': name}):
	parent = parents_collection.find_one({'name': name})
	# 후에 아이들 삭제에 사용하기 위한 id 저장
	parent_id = parents_collection.find_one({'name': name}, {"_id": 1})
	# parents 컬렉션에서 삭제
	delete_parent = parents_collection.delete_many({'name': name})
	
	child_id = list(children_collection.find({'parent_id': parent_id}, {"_id": 1}))
	# 아이들의 행동 분석 결과 삭제
	for ids in child_id:
	ress = results_collection.delete_many({'child_id': ids})
	# 어린이 컬렉션에 해당 부모 id 가진 아이들 모두 삭제
	delete_children = children_collection.delete_many({'parent_id': parent_id})
	
	# 해당 이름이 선생님 컬렉션에 존재한다면
	elif teachers_collection.find_one({'name': name}):
	# 선생님 컬렉션에서 삭제
	delete_teacher = teachers_collection.delete_many({'name': name})

모듈명	urls.py
입력 자료	없음
출력 자료	해당 url에 맞는 페이지로 이동
기능	Django에서 views.py 기능을 사용하기 위해선 무조건 사용해야 하는 모듈.
호출모듈	views.py, 필요하다면 알맞은 웹
모듈의 상세 설계	urlpatterns = [path('', views.main, name='main'), # 첫 페이지(로그인 페이지) path('login/', views.login_user, name='login_user'), # 로그인 페이지 path('signup/', views.add_users, name='add_users'), # 회원가입 페이지 및 회원가입 처리 # 선생님 페이지

	path('teachers/', views.teachersPage, name='teachers_page'), # 로그아웃 path('logout/', views.logout_view, name='logout'), # 알림장 내용 확인 path('api/showNotice_cont/<str:id>/', views.showNotice_cont, name='showNotice_cont'), # 알림장 쓰기 path('api/writeNotice/<str:teacher_id>/<str:classroom>/', views.writeNotice, name='writeNotice'), # 알림장 모든 내용 가져오기 path('api/get_notice_logs/<str:id>/', views.get_notice_logs, name='get_notice_logs'), # 회원 탈퇴용 path("bye/<str:name>/", views.withdrawalUser, name='withdrawalUser'), # 자녀 추가용 path('addchild/', views.add_child, name='add_child'),]

모듈명	MongoDB Atlas
입력 자료	행동 분석 결과, 알림장 정보, 어린이 ID, 교사 ID
출력 자료	MongoDB에 데이터 삽입, 출력값 없음
기능	AI 서버 또는 UI에서 전달한 내용을 MongoDB Atlas에 저장하는 기능
호출모듈	Flask 서버의 app.py, 알림장 작성 라우트에서 호출
모듈의 상세 설계	# 전달받은 데이터의 구조에 따라 저장할 컬렉션 결정 - 행동 분석 결과인 경우 -> ai_results - 알림장 내용인 경우 -> notice # 공통적으로 MongoDB Atlas의 graduation_work DB에 접속 # 각 컬렉션에 insert_one()으로 저장 수행

3) UI

모듈명	createId_page.html
입력 자료	회원 정보(id, pw, 역할, 이름, 전화번호, 반(선생님일 때))
출력 자료	서버로 데이터 POST
기능	서버를 통해 데이터베이스에 정보를 저장하기 위한 POST 요청 보냄
호출모듈	form에서 POST하면서 add_users의 url 호출
모듈의 상세 설계	<form method="POST" action="{% url 'add_users' %}"> {% csrf_token %} 선생님인지 학부모인지 선택할 수 있도록 라디오 버튼 존재 아이디, 비밀번호, 비밀번호 확인, 이름, 전화번호를 입력할 수 있는 input 존재 <!-- 선생님 필드 --> (선생님일 때만(라디오 버튼에서 선생님 클릭햇을 때 열리는 부분) 담당 반을 선택할 수 있도록 함. <!-- 이 버튼을 누르면 add_user로 이동하여 저장 로직 실행 --> <button type="submit" class="createId-btn">저장</button> </form>

모듈명	main.html
입력 자료	회원 정보(id, pw, 역할)
출력 자료	서버로 데이터 POST
기능	서버를 통해 데이터베이스에 존재하는지 확인하고 로그인을 하기 위한 POST 요청 보냄
호출모듈	form에서 POST하면서 login_user의 url 호출
모듈의 상세 설계	<form method="POST" action="{% url 'login_user' %}"> {% csrf_token %} 학부모인지 선생님인지 확인하기 위한 라디오 버튼 존재 아이디, 비밀번호 입력 칸 존재

	<!-- 이 버튼을 누르면 login_user로 이동하여 로그인 로직 실행 --> <button type="submit" class="login-btn" >로그인</button> </form> <!-- 계정을 만들고 싶을 경우 회원가입 창으로 이동할 수 있는 링크 --> 계정이 없으신가요? 회원가입

모듈명	teachers_page.js
입력 자료	
출력 자료	서버로 데이터 POST
기능	어린이 개개인의 알림장을 열었을 때 html 디자인 변경을 위함. 열린 알림장의 어린이 ID를 전역 변수에 저장한 뒤 개인 알림장 내용을 저장할 때 사용함. 개인 알림장 내용 저장할 땐 AJAX를 사용함.
호출모듈	views.py와 해당 html

모듈의 상세 설계	// 개인 알림장 열기
	let currentChildId; // 현재 열려있는 알림장의 아이 ID (나중에 알림장 내용할 때 쓸 것)
	
	function openForm(childName, childId) {
	document.querySelector("#formArea .form-content h3").innerText = `${childName}의 알림장 내용 작성`;
	currentChildId = childId; // 현재 열려있는 아이의 ID 저장
	};
	
	// 개인 알림장 내용 저장 (AJAX)
	document.addEventListener("DOMContentLoaded", function () {
	const form = document.getElementById("writeNotice");
	if (form) {
	form.addEventListener("submit", function(e) {
	e.preventDefault(); // 폼 기본 동작(페이지 이동) 막기
	
	const content = document.getElementById("noticeContent").value;
	const csrfToken = document.querySelector("[name=csrfmiddlewaretoken]").value;
	const url = form.dataset.url; // form 요소의 data-url 속성 값 읽기
	
	fetch(url, {

```
            method: "POST",
            headers: {
              "Content-Type": "application/x-www-form-urlencoded",
              "X-CSRFToken": csrfToken
            },
            body: new URLSearchParams({
              content : content,
              child_id: currentChildId  // 현재 열려있는 아이의 ID 전달
            })
          })
          .then(res => {
            if (!res.ok) {
              // 응답 상태가 200 OK가 아니면
              throw new Error('서버 오류: ${res.status}');
            }
            return res.json();
          })
          .then(data => {
            if (data.error) {
              alert("데이터 에러" + data.error);
            } else {
              alert("저장되었습니다!");
              form.querySelector('textarea').value = '';  // 저장 후 textarea
초기화
            }
          })
          .catch(err => {
            alert('저장 중 오류 발생: ${err.message}');
            console.error(err);
          });
        });
    }
});

// 개인 알림장 내용 삭제(textarea 비우기)
function clearForm() {
    const formArea = document.getElementById('formArea');
    formArea.querySelector('textarea').value = '';
}
```

모듈명	parents_page.js
입력 자료	

출력 자료	서버로 데이터 POST
기능	학부모가 자녀들 각각의 알림장과 오늘의 하루를 볼 수 있도록 팝업 열고 닫는 기능 존재. 사진과 영상을 넣었을 때 슬라이드로 볼 수 있도록 하는 기능 존재.
호출모듈	views.py와 해당 html

모듈의 상세 설계	// 알림장 팝업 띄우기
	function openPop(childName, childId) {
	document.getElementById(“popup_layer“).style.display = “block“;
	document.querySelector(“#popup_layer h2“).innerText = ‘${childName}의 알림장‘;
	fetch(‘/api/showNotice_cont/${childId}/‘)
	.then(response => response.json())
	.then(data => {
	if (data.content) { // 값이 있을 때만
	document.querySelector(“#popup_layer p“).innerText = data.content;
	console.log(“data.content:“, data.content);
	}
	else {
	document.querySelector(“#popup_layer p“).innerText = ‘아직 알림장이 작성되지 않았습니다□‘;
	}
	console.log(“data.total_res:“, data.total_res);
	
	document.querySelector(“#totalRes“).innerText = ‘* 오늘의 행동 감지: ${data.total_res}건‘;
	Object.keys(data.event_counts).length);
	if (data.event_counts) {
	const rows = document.querySelectorAll(“#eventTable tbody tr“);
	
	for(const [event, count] of Object.entries(data.event_counts)) {
	for(const row of rows) {
	const th = row.querySelector(“th“);
	const td = row.querySelector(“td“);
	if(th && th.innerText.trim() === event) { // 이벤트 이름과 일치하면
	td.innerText = ‘${count}건‘; // 같은 인덱스 td에 값 넣기
	break; // 다음 이벤트로 넘어가기

```
                        }
                    }
                }
            }
        })
        .catch(error => console.error('Error: ', error));
}

// 알림장 팝업 닫기
function closePop() {
    document.getElementById("popup_layer").style.display = "none";
}

// 오늘의 하루 팝업 띄우기
function openTodayPop(childName) {
    document.getElementById("today_popup_layer").style.display = "block";
    document.querySelector("#today_popup_layer h2").innerText =
`${childName}의 하루`;
}
// 오늘의 하루 팝업 닫기
function closeTodayPop() {
    document.getElementById("today_popup_layer").style.display = "none";
}

// 사진/영상 슬라이드1
document.addEventListener('DOMContentLoaded', function () {
    const prev = document.querySelector('.prev'); // 이전 이미지
    const next = document.querySelector('.next'); // 다음 이미지
    const slideBox = document.querySelector('.slide_box');
    const slideItems = document.querySelectorAll('.slide_item');
    const slideLength = slideItems.length;
    let currentIndex = 0;

    const moveSlide = function (num) {
        slideBox.style.transition = 'transform 0.4s ease'; // 부드럽게
        slideBox.style.transform = `translateX(${-num * 280}px)`;
        currentIndex = num;
    };

    // 이미지에 클릭 이벤트 연결
    prev.addEventListener('click', () => {
        if (currentIndex > 0) {
            moveSlide(currentIndex - 1);
```

```
        }
    });

    next.addEventListener('click', () => {
        if (currentIndex < slideLength - 1) {
            moveSlide(currentIndex + 1);
        }
    });
});

// 사진/영상 슬라이드2
document.addEventListener('DOMContentLoaded', function () {
    const prev = document.querySelector('.prev1'); // 이전 이미지
    const next = document.querySelector('.next1'); // 다음 이미지
    const slideBox = document.querySelector('.slide_box1');
    const slideItems = document.querySelectorAll('.slide_item1');
    const slideLength = slideItems.length;
    let currentIndex = 0;

    const moveSlide = function (num) {
        slideBox.style.transition = 'transform 0.4s ease'; // 부드럽게
        slideBox.style.transform = `translateX(${-num * 280}px)`;
        currentIndex = num;
    };

    // 이미지에 클릭 이벤트 연결
    prev.addEventListener('click', () => {
        if (currentIndex > 0) {
            moveSlide(currentIndex - 1);
        }
    });

    next.addEventListener('click', () => {
        if (currentIndex < slideLength - 1) {
            moveSlide(currentIndex + 1);
        }
    });
});
```

4) 인공지능서버

모듈명	app.py

입력 자료	라즈베리파이에서 5분 단위로 전송된 .mp4 영상 파일
출력 자료	HTTP JSON 응답
기능	Flask 기반 웹 서버를 구동하며 /upload API를 통해 동영상을 수신함.- 분석 요청 실행 및 결과 반환
호출모듈	predict.py, save_result.py

모듈의 상세 설계	# 서버 확인용 기본 라우트
	@app.route('/')
	def home() :
	return "AI 서버가 동작 중입니다!"
	
	# 영상 업로드를 처리하는 API 엔드포인트
	@app.route('/upload', methods=['POST'])
	def upload_video():
	try:
	# 라즈베리 파이로 전송된 파일 객체 추출
	file = request.files['video']
	# 업로드된 파일의 원래 이름
	filename = file.filename
	# 저장 경로 설정
	save_path = os.path.join(UPLOAD_FOLDER, filename)
	# 파일 저장
	file.save(save_path)
	# 분석 및 CSV 저장
	csv_path = analyze_video_and_save_csv(save_path)
	# MongoDB 저장
	save_csv_to_mongo(csv_path)
	# 성공 응답 반환
	return jsonify({"status": "success", "msg": f"{filename} 처리 완[>
	# 오류 발생 시 에러메세지 반환
	except Exception as e:
	return jsonify({"status": "error", "msg": str(e)}), 500

모듈명	save_result.py
입력 자료	분석 결과가 담긴 .csv 파일 경로
출력 자료	MongoDB에 저장된 분석 결과 Document
기능	분석 결과가 저장된 .csv 파일을 열어 row 단위로 읽은 뒤, 각 row를 MongoDB document로 변환하여 저장하는 역할 수행
호출모듈	predict.py, save_result.py
모듈의	def save_csv_to_mongo(csv_path):

상세 설계	#CSV 파일 읽고 MongoDB에 저장하는 함수
	with open(csv_path, newline='', encoding='utf-8') as csvfile:
	reader = csv.DictReader(csvfile)
	for row in reader:
	doc = {
	# 아이 식별자
	"child_id": row["child_id"],
	# 분석된 행동 분류
	"action": row["action"],
	"timestamp": datetime.strptime(row["timestamp"], "%Y-%m-%d %H:%M:%S")
	}
	
	# MongoDB에 한건씩 insert
	collection.insert_one(doc)

3.5 테스트

1) 테스트 절차

(1) Raspberry Pi

① 카메라 프레임은 0.5 초 이내 지연으로 노트북 화면에 표시돼야 한다.

· 테스트 결과, 타임스탬프 기준 평균 0.31 초 지연임을 확인함

② 네트워크가 10 초 끊겨도 5 초 안에 자동 재연결돼야 한다.

· LAN 케이블 제거-복구 시험에서 4.1 초 만에 재접속 후 전송 재개됨을 확인함

③ 5 분 단위 MP4 세그먼트 길이는 300 ± 2 초 범위여야 한다.

· 3 세그먼트 녹화 후 ffprobe 결과가 300.2 / 299.8 / 300.0 초로 기준 충족

④ 세그먼트 업로드 성공률은 98 % 이상이어야 한다.

· 10 세그먼트 연속 시험에서 **10 / 10 세그먼트 업로드 성공(100 %)**을 확인함

⑤ 전원 차단 후 재부팅 시 1 분 내 자동 녹화·업로드가 재개돼야 한다.

· 재부팅 실험에서 47 초 만에 video_uploader 서비스가 시작되고 로그가 기록됨을 확인함.

(2) 서버

① 웹페이지에서 입력된 데이터가 서버를 통해 데이터베이스에 정확히 저장되는지를 검증함.

② 클라이언트(웹페이지)로부터의 요청이 서버에 정상적으로 도달하고, 그에 따른 응답이 클라이언트에 적절히 전달되는지를 확인함.

③ 웹페이지 및 JSON 형식의 데이터가 브라우저에 이상 없이 렌더링되는지를 점검함.

(3) UI

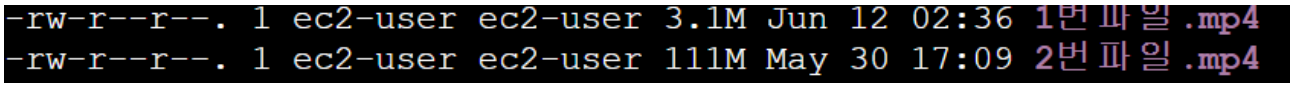
① 사용자의 입력을 기반으로 회원가입, 로그인, 자녀 정보 추가, 알림장 작성 등의 기능이 정상적으로 동작하는지 확인함.

② 사용자 역할(학부모, 선생님)에 따라 적절한 데이터베이스 컬렉션의 정보가 정확히 불러와지는지를 검증함.

③ HTML 요소가 JavaScript에 의해 동적으로 변경되어야 하는 상황에서 화면이 올바르게 갱신되는

지를 확인함.
④ 서버로부터 전달된 웹페이지가 정상적으로 렌더링되며, 데이터베이스의 값이 왜곡되거나 비정상적으로 출력되지 않는지를 점검함.
⑤ 선생님 페이지에서 작성된 알림장이 해당 자녀의 학부모 페이지에 정확히 표시되는지를 확인함.

(4) AI Server
① 라즈베리파이에서 보낸 영상이 서버 내부에 정상적으로 저장되는지 확인함.
② 행동분석 결과를, csv형식으로 변환하여 데이터베이스에 손실 없이 전송되는지 확인함.
③ 테스트 결과

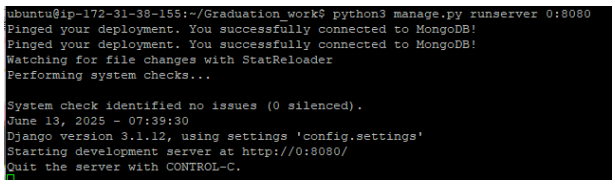
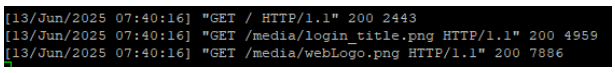
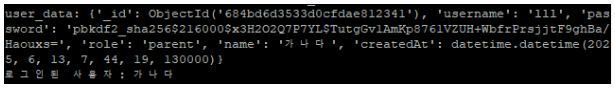
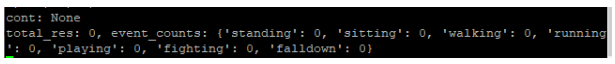


- 라즈베리파이에서 보낸 영상 저장 확인

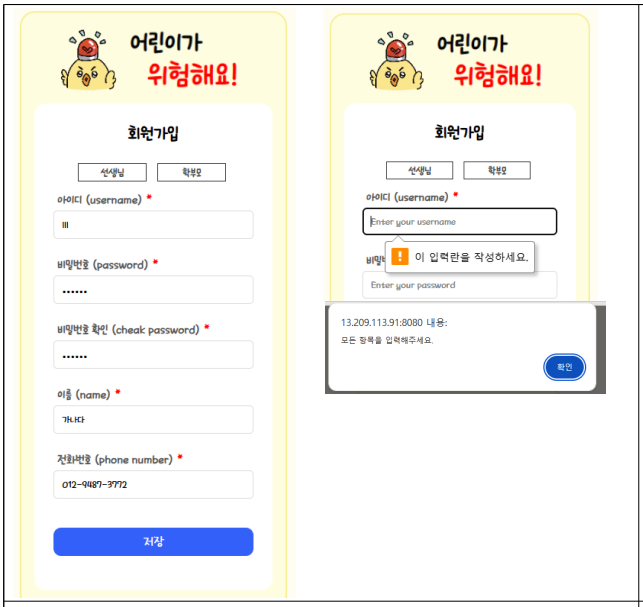
(5) AI
① 라벨링 데이터를 가지고 만든 모델을 적용하여 map와 loss가 얼마나 나오는지 확인, 첫 테스트 결과 모델의 정확도가 72%정도 나오는 것을 확인함. 또한 loss데이터를 확인하여 오류값이 있는지 확인하고 데이터를 제거 또는 교체하여 진행함.
② 이후 이미지와 영상 데이터를 더 추가해 한번더 학습시켜 각각 test,valid 데이터셋에서 loss값이 얼마나 줄어드는지 확인하고 다시 테스트 하여 76%정도 추가로 나오는 것을 확인함.
③ 모델과 작성한 코드를 서버에 전송하여 영상 뿐만 아니라 실시간영상으로 진행하여 결과값이 나오는지 확인함. 결과값을 .csv파일로 저장하여 AI서버에서 DB서버로 전송되는 것을 확인함.

2) 테스트 결과

(1) 서버

<table>
<tr><td> 
</td><td> 
</td></tr>
<tr><td>(a) Django로 서버 실행 성공 확인</td><td>(b) 홈페이지 접속 시 렌더링 확인</td></tr>
<tr><td> 
</td><td> 
</td></tr>
<tr><td>(c) 로그인 시 user 데이터 확인</td><td>(d) 알림장에 전달되는 정보 확인</td></tr>
</table>

(2) UI



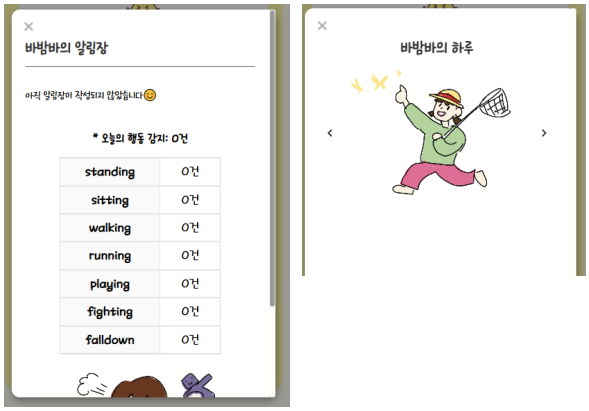
(a) 회원가입 조건 검사 확인



(b) 학부모 페이지에서 자녀 추가 여부 확인

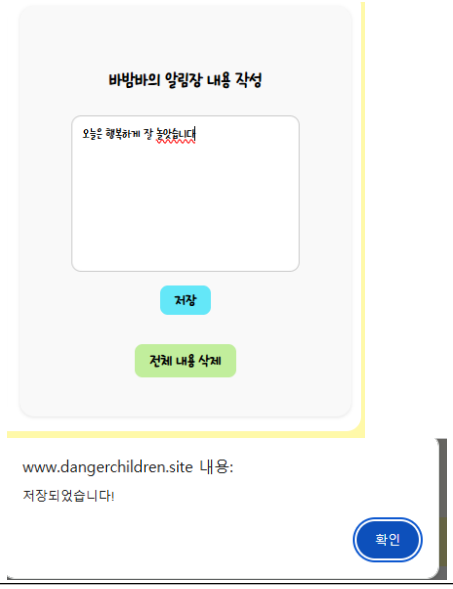


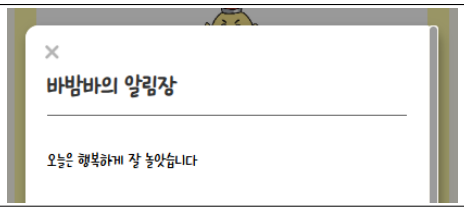
(c) 자녀 존재 시 리스트에 뜨는지 확인



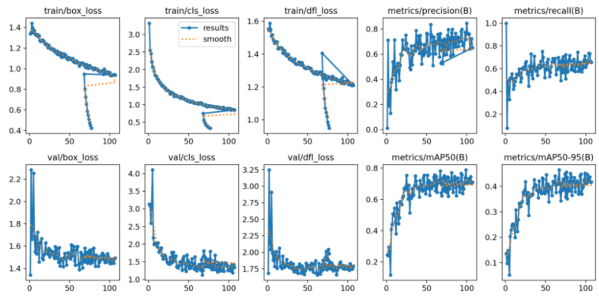
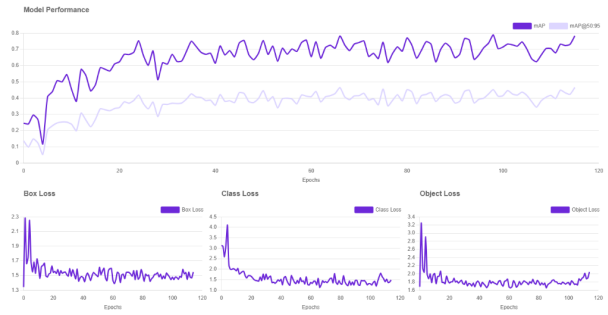
(d) 알림장과 오늘의 하루 페이지에 데이터 확인



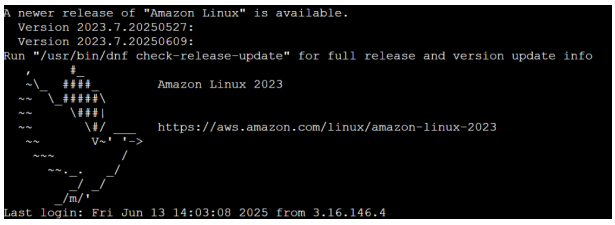
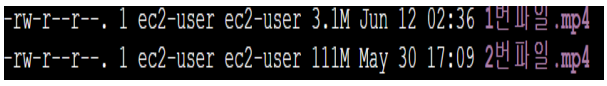
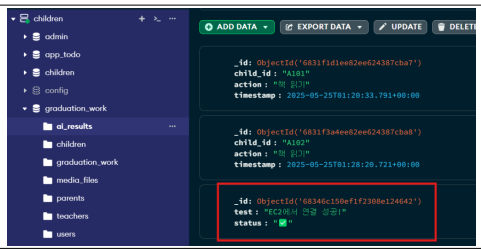


	
(e) 추가된 자녀 해당 반 리스트에서 확인	(f) 알림장 저장 시 학부모 페이지에서 확인 가능
	
(g) 회원 탈퇴시 로그인 안 됨 확인	

(3) AI

	
(a) 정확도	(b) 바운딩, 클래스, 객체 손실함수

(4) AI server

	
(a) Flask로 서버 실행 성공 확인	(b) 라즈베리파이에서 보낸 영상이 서버에 저장됨을 확인
	
(c) Flask 서버와 DB 연결 확인	

3.6 결과 도출

1) Raspberry Pi

① 실시간 스트리밍 지연은 0.5 초 이내여야 한다.

• 테스트에서 프레임 타임스탬프를 비교한 결과, 평균 지연 312 ms로 목표를 만족함.

② 5 분 단위 MP4 녹화-업로드 성공률은 98 % 이상이어야 한다.

• 10 세그먼트 반복 시험에서 **10 / 10 세그먼트 업로드 성공(100 %)**을 확인함.

③ 네트워크가 10 초 끊겨도 5 초 이내 자동 재연결 해야 한다.

• LAN 케이블을 제거-복구한 실험에서 4.1 초 만에 재접속 후 전송 재개를 확인함.

④ 부팅 후 1 분 안에 자동 녹화-업로드가 재개되어야 한다.

• 전원 차단-재부팅 시험 결과, 47 초 만에 video_uploader.service가 실행되고 업로드 로그가 시작됨을 확인.

2) 서버

① 서버는 오류 없이 상시 구동되어야 한다.

• 테스트 과정을 거치며 발생한 오류들을 수정했기 때문에 현재 오류 없이 상시 구동이 가능함.

② 데이터 송수신 간 데이터 손실이 없어야 하며, 값을 웹페이지에 제대로 전달되어야 한다.

• 테스트 결과 UI 표 (f)에서 저장된 내용이 온전히 저장되는 것을 확인함. 다른 데이터들도 마찬가지임.

3) UI

① 사용자들이 직관적인 UI 환경으로 편리하게 사용 가능하도록 해야 한다.

• 회원가입, 로그인 시 직관적인 화면 구성과 버튼으로 쉽게 사용 가능하도록 함.

• 예외 상황(입력 칸이 비거나, 역할 선택이 되지 않았을 경우)에 alert나 HTML에서 input 사용 시 required로 알려주는 방식 사용함.

② 회원 탈퇴를 할 경우엔 한 번 더 물어봐야 함.

• 실제로 alert로 확인 버튼을 한 번 더 눌러야 실행 됨.

4) AI

① 실시간으로 들어오는 영상에 대하여 객체를 정확하게 인지할 수 있어야 함.

• 객체가 움직임에 따라 x좌표 또는 y좌표 값이 변하는 경우 함께 변하며 인식하는 것을 확인했다.

② 분류한 행동을 실시간으로 반영하여 DB에 보내야 한다.

• 재저장되는 시간을 10초로 설정하여 최대한 공백기간을 줄여 업데이트한다. 저장된 파일을 서버로 보내 저장되는 것을 확인함.

4. 결과물에 대한 기대효과 및 활용방안

1) 기대효과:

- AI가 아동의 행동을 분석하여 뛰거나 넘어짐 등의 이상 행동을 감지하여 교사가 대처할 수 있음.
- 알림장 기능을 통해 교사의 행정 업무의 효율성을 높임.
- AI와 데이터 기반의 체계적인 아동 관리 시스템을 도입하여 보육시설에 대한 학부모의 신뢰도가 향상됨.

2) 활용방안

- 아동별 행동 패턴을 분석하여 맞춤형 케어를 제공함으로써 특정 아동에 대한 개별 관리가 가능.
- 아동 학대 및 안전사고를 빠르게 대처 가능.

5. 작품 수행 후기 및 개선 방향

1) 느낀점

- 실제 테스트 중 Wi-Fi가 잠시 끊겼는데, 자동 재연결·재시도 로직을 미리 넣어두지 않았다면 영상이 완전히 끊겼을 상황을 “예외 처리가 곧 신뢰성”이라는 것을 경험함.
- 녹화, 변환, 업로드를 서로 다른 스레드로 나누고 큐로 연결하면서 프로듀서-컨슈머 패턴을 실습을 겪어볼 수 있었음.
- AI 팀이 10 초 단위로 분석하겠다고 방향을 바꿔도 제가 정의한 “MP4 5 분 업로드” 인터페이스는 그대로 유지할 수 있었음. 초기에 ‘역할 경계’를 명확히 정해 둔 덕분에 서로 코드를 뜯지 않고도 연동이 가능했기에 다음 프로젝트 수행 시 좀 더 나은 성과를 기대함.
- 익숙하지 않은 프로그램을 사용하고, 깊이 있게 알고 있지 않은 프로그래밍 언어을 쓰다 보니 다양한 오류를 맞닥뜨리고 해결에 많은 시간을 쓰는 상황을 겪게 됐음. 하지만 이 상황이 있음으로써 발전할 수 있는 계기가 되었다고 생각하고, 실제로 해결하는 과정에서 이론적으로 학습하는 것보다 오래 기억에 남게 됐음.
- 팀원끼리 회의를 통해 각 파트에서 필요한 기능이 무엇인지 피드백하는 시간도 갖고, 힘든 부분은 서로 공유하며 팀워크란 게 뭔지 알게 되는 시간이었음.
- 프로젝트를 진행하며 여러 사람과의 협업과 소통이 어려움을 느낌.
- 새로운 프로그램을 이용하여 새로운 방식으로 진행하다보니 오류와 문제를 많이 겪음.
- 하나의 기능을 구성하고 하나씩 기능을 추가하려 하였지만 추가하며 계속하여 오류가 생겨나 계속 진행할 수 없는 부분에서 답답함을 느낌.
- 여러 대체 방안과 해결법을 사용해보며 문제상황을 해결할 수 있는 능력을 기를 수 있었음.
- 팀원들과 소통하며 내가 맡은 파트 뿐만 아닌 팀원들의 파트까지 이해하여 좀 더 수월하게 필요한 부분을 알 수 있어서 소통의 중요성을 느낄 수 있었음.
- 팀원들의 일정을 맞춰 회의하고 소통하는 것이 어려운 것을 느낌. 이로 인해 의사소통의 중요성을 느끼고 갈등의 소재가 없도록 하였음.
- 각자 파트별 진행 상황을 파악하고 어려운 부분을 해결할 수 있는 방법 습득함.

2) 개선사항

(1) UI 디자인 개선

• 생각보다 투박한 디자인에 행동 분석 결과를 확인할 때 직관적이지 않은 것 같다고 생각함. 따라서 직관적인 분석 결과를 위한 그래프와 표 사용을 할 필요가 있음.

- 모델 정확도 향상.
- 카메라를 추가하여 사각지대 최소화

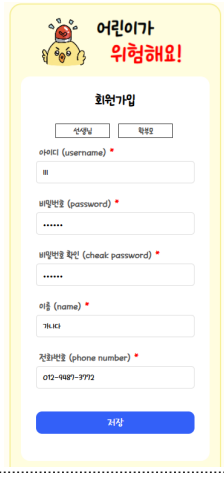
6. 팀원 역할분담

No.	성명	담당	수행역할	참여율
1	신은진	팀장	프로젝트 총괄 및 매니저, 라벨링 작업	100%
2	이예은	팀원	데이터베이스 구축 및 관리, 라벨링 작업, 인공지능 서버 제작	100%
3	안채은	팀원	AI담당, 라벨링 및 모델 개발, 테스트	100%
4	장슬기	팀원	백엔드및 프론트엔드 서버 개발, UI 개발, 라벨링 작업	100%
5	황인정	팀원	Raspberry Pi 캠 연결 및 캠 서버 개발	100%

7. 작품 수행 예산

항 목	금액(원)	지출내역	비 고
재료비	285,010	라즈베리파이5 8GB: 119,500원 1개 = 119,500원 라즈베리파이 HQ 카메라모듈 C/CS mount: 72,000원1 개 = 72,000원 라즈베리파이 HQ 카메라 모듈용 6mm 광각 렌즈 3MP: 37,000원 1개 = 37,000원 Arducam Tripod for Raspberry Pi High Quality Camera: 12,100원 X 2개 = 24,200원	배송비: 32,310
회의비	38,100	6/2 회의비: 신은진 외 3명(써브웨이 마산경남대점)	
여비			
수용비			
문헌구입비			
합 계	**323,110**		

8. 작품 수행 사진 및 실행 장면



회원가입 조건 검사



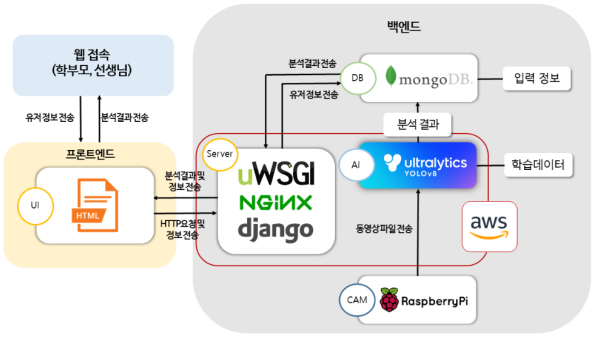
자녀 추가 여부



해당 반에 아이들 리스트



싸우기와 서있는 객체 인식



시스템 구조



앉아있는 것과 서 있는 객체 인식

9. 참고 문헌

[1] '하늘이 사건'으로 만들어진 '하늘이법' 내용
https://blog.naver.com/pure082/223765293410
[2] '하늘이 사건' 정리
https://blog.naver.com/PostView.naver?blogId=artivenews&logNo=223793018257
[3] 김하늘양의 유가족 기사
https://www.yna.co.kr/view/AKR20250210149451063
[4] AWS EC2에 깃허브 이용해 장고 어플리케이션 배포하기,
https://loann.tistory.com/51
[5] 우분투에서 MongoDB 설치하기,
https://www.cloudbooklet.com/developer/how-to-install-mongodb-on-ubuntu-22-04
[6] pymongo로 DB 연결하기, https://he-kate1130.tistory.com/42
[7] MongoDB 쿼리 명령어, https://minimilab.tistory.com/43
[8] MongoDB와 Django 연결하기, https://dong5854.tistory.com/13
[9] HTML 웹페이지 콘텐츠 섹션 나누기, https://parkjh7764.tistory.com/46
[10] 스크롤바 생성, https://codingbroker.tistory.com/127
[11] 비디오 띄우기, https://videojs.com/getting-started/
[12] 자바스크립트로 팝업창 띄우기, https://lktprogrammer.tistory.com/127
[13] 그래프 띄우기, https://www.highcharts.com/download/#nav-dashboards-tab
[14] 우분투 22.04 Let's encrypt 인증서 발급받기,
https://blog.naver.com/e_view/223254759642
[15] 도서 - 이한영의 Django 입문(이한영, 디지털북스)

10. 프로젝트 국문 및 영문 요약

기존 보육 시설에서는 교사가 직접 아이를 관찰하여 위급 상황을 인지해야 하지만, 본 프로젝트에서는 AI 모델을 활용하여 실시간으로 어린이의 행동을 분석하고 감지할 수 있다. 이를 통해 교사의 업무 부담을 줄이는 동시에, 사고 발생 시 즉각적인 대응이 가능한다. 일반적인 어린이집과 유치원에서는 하루 일과가 끝난 후 부모에게 아이의 활동을 요약하여 전달하는 방식이지만, 본 프로젝트는 실시간 알림 시스템을 도입하여 차별성을 가짐. 또한 하루 일과가 끝난 후 교사가 작성하는 알림장을 AI 분석 데이터와 결합하여 보다 신뢰성 높은 정보를 부모에게 제공함. 이를 통해 부모는 아이의 하루 생활을 보다 정확하고 객관적으로 파악할 수 있다.

In existing childcare facilities, teachers must monitor the child to be aware of an emergency, but in this project, AI models can be used to analyze and detect children's behavior in real time. This reduces the burden on teachers and enables immediate

response in the event of an accident. In general daycare centers and kindergartens, children's activities are summarized and delivered to parents after a day's work is over, but this project is differentiated by introducing a real-time notification system. In addition, more reliable information is provided to parents by combining notifications written by teachers after the day's work with AI analysis data. Through this, parents can more accurately and objectively grasp the child's daily life.

11. 최종 결과물 첨부 자료

[1] 백엔드 & UI
https://github.com/opeach2v/Graduation_project

- 프로젝트 소스코드(주석 포함)은 분량이 많아 별도로 첨부하였습니다.